

# Kazakh-British Technical University

Databases

Laboratory work № 4

04.10.2026

Almaty

## 1. Retrieve all airline names in uppercase.

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, with the 'airline' table selected under 'Tables (10)'. The table has six columns: 'airline\_id' (integer), 'airline\_code' (character varying(50)), 'airline\_name' (character varying(50)), 'airline\_country' (character varying(50)), 'created\_at' (date), and 'update\_at' (date). The right pane shows a SQL query: `SELECT UPPER(airline_name) as airline_name from airline`. The 'Data Output' pane displays the results of the query, showing the 'airline\_name' column with values: IPC, PDN, KLE, KHS, YLQ, NGL, 0, QIG, and NQX. The status bar indicates 'Total rows: 50' and 'Query complete 00:00:00.054'.

| airline_name |
|--------------|
| IPC          |
| PDN          |
| KLE          |
| KHS          |
| YLQ          |
| NGL          |
| 0            |
| QIG          |
| NQX          |

## 2. Replace any occurrence of the word "Air" in airline names with "Aero".

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, with the 'airline' table selected under 'Tables (10)'. The table has six columns: 'airline\_id' (integer), 'airline\_code' (character varying(50)), 'airline\_name' (character varying(50)), 'airline\_country' (character varying(50)), 'created\_at' (date), and 'update\_at' (date). The right pane shows a SQL query: `SELECT REPLACE(airline_name, 'Air', 'Aero') as airline_name from airline`. The 'Data Output' pane displays the results of the query, showing the 'airline\_name' column with values: IPC, PDN, KLE, KHS, YLQ, NGL, 0, QIG, and NQX. The status bar indicates 'Total rows: 50' and 'Query complete 00:00:00.065'.

| airline_name |
|--------------|
| IPC          |
| PDN          |
| KLE          |
| KHS          |
| YLQ          |
| NGL          |
| 0            |
| QIG          |
| NQX          |

3. Find all flight numbers that coordinates with both airline 1 and airline 2.

The screenshot shows the pgAdmin 4 interface. The left pane displays the 'Object Explorer' with the 'flights' table selected under the 'public' schema. The right pane shows the 'Query' editor with the following SQL query:

```
1 SELECT flight_no, COUNT(*) from flights
2 where airline_id in (1, 2)
3 group by flight_no
4 HAVING COUNT(DISTINCT airline_id) = 2
5
```

The 'Data Output' pane at the bottom shows the query results with two columns: 'flight\_no' (character varying(50)) and 'count' (bigint). The status bar indicates 'Total rows: 0' and 'Query complete 00:00:00.086'.

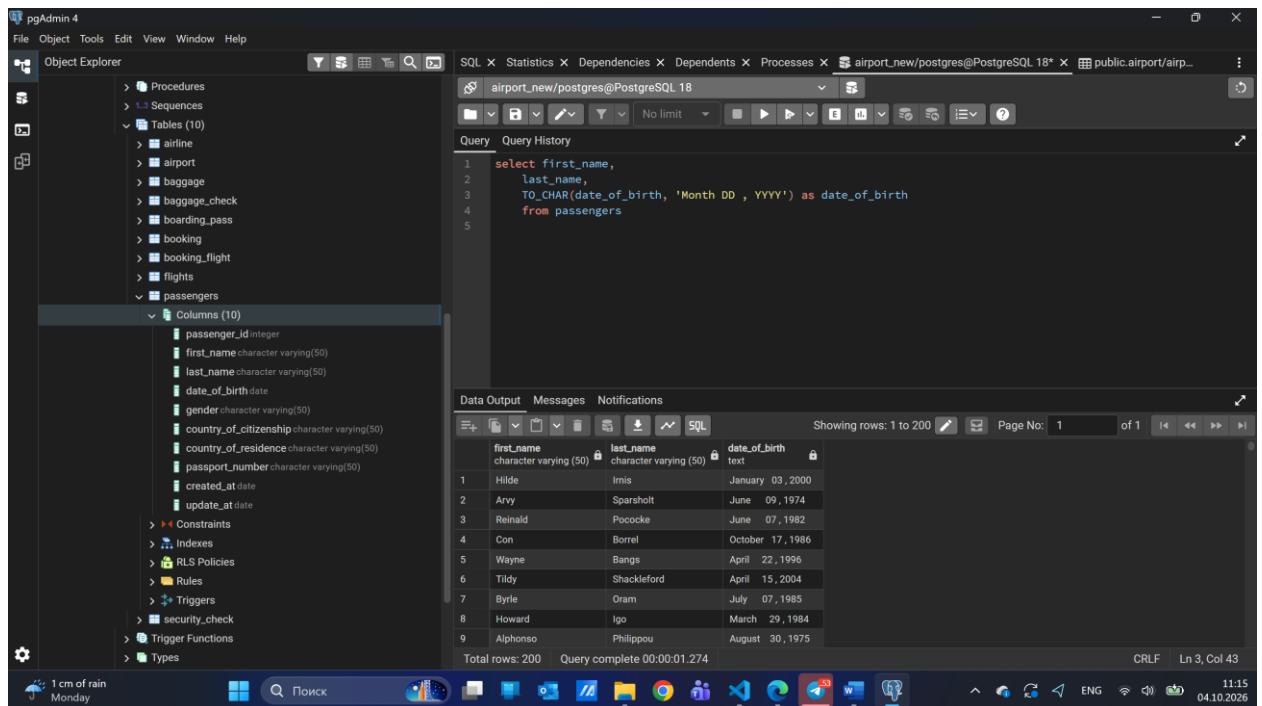
4. Retrieve airports that contain the word "Reginal" and "Air" in their names.

The screenshot shows the pgAdmin 4 interface. The left pane displays the 'Object Explorer' with the 'airport' table selected under the 'public' schema. The right pane shows the 'Query' editor with the following SQL query:

```
1 Select * from airport
2 where airport_name like '%Reginal%'
3 and airport_name like '%Air%'
```

The 'Data Output' pane at the bottom shows the query results with columns: 'airport\_id' (integer), 'airport\_name' (character varying(50)), 'country' (character varying(50)), 'state' (character varying(50)), 'city' (character varying(50)), 'created\_at' (date), and 'update\_at' (date). The status bar indicates 'Total rows: 0' and 'Query complete 00:00:00.067'.

## 5. Retrieve passenger names and format their birth dates as 'Month DD, YYYY'.



The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including the 'passengers' table with columns like 'passenger\_id', 'first\_name', 'last\_name', and 'date\_of\_birth'. The central pane shows a SQL query:

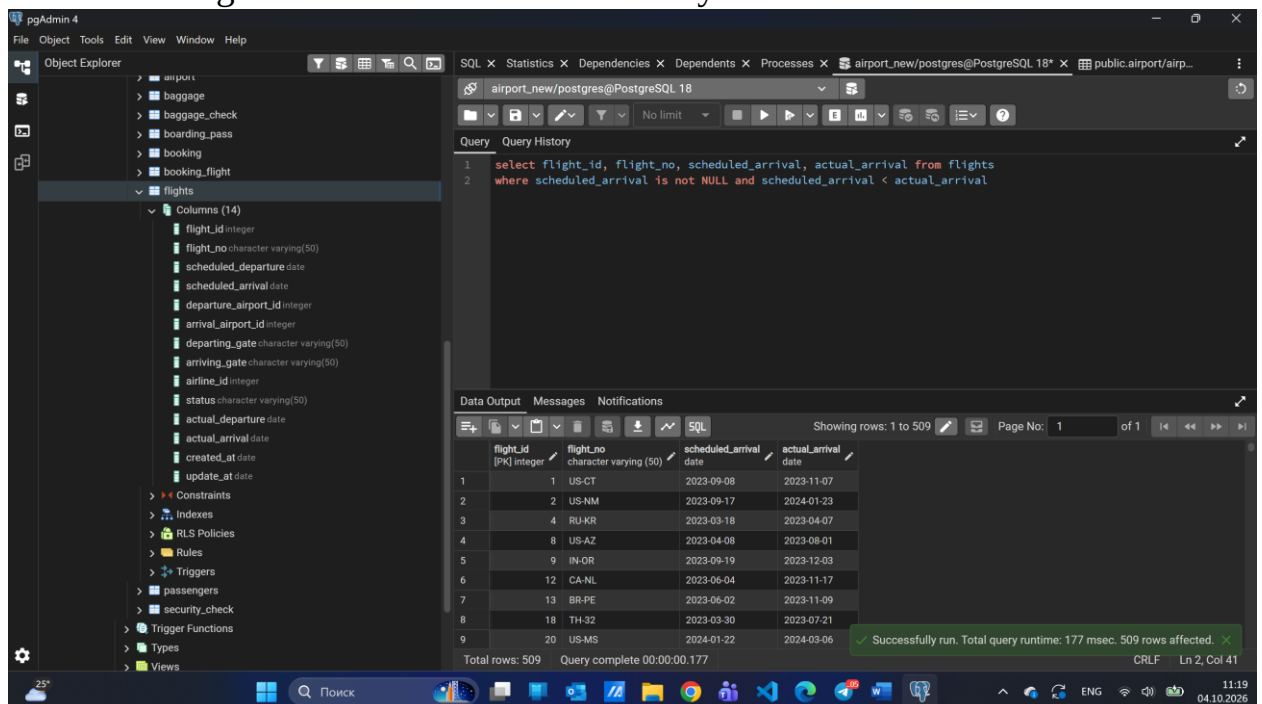
```
1 select first_name,
2       last_name,
3       TO_CHAR(date_of_birth, 'Month DD, YYYY') as date_of_birth
4 from passengers
5
```

The right pane displays the query results in a table format, showing the first 9 rows of data:

| first_name | last_name   | date_of_birth    |
|------------|-------------|------------------|
| Hilde      | Imis        | January 03, 2000 |
| Ary        | Sparsholt   | June 09, 1974    |
| Reinald    | Pococke     | June 07, 1982    |
| Con        | Borrel      | October 17, 1986 |
| Wayne      | Bangs       | April 22, 1996   |
| Tildy      | Shackleford | April 15, 2004   |
| Byrle      | Gram        | July 07, 1985    |
| Howard     | Igo         | March 29, 1984   |
| Alphonso   | Philippou   | August 30, 1975  |

The status bar at the bottom indicates 'Total rows: 200' and 'Query complete 00:00:01.274'.

## 6. Find flight numbers that have been delayed based on the actual arrival time.



The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including the 'flights' table with columns like 'flight\_id', 'flight\_no', 'scheduled\_arrival', and 'actual\_arrival'. The central pane shows a SQL query:

```
1 select flight_id, flight_no, scheduled_arrival, actual_arrival from flights
2 where scheduled_arrival is not NULL and scheduled_arrival < actual_arrival
```

The right pane displays the query results in a table format, showing the first 9 rows of data:

| flight_id | flight_no | scheduled_arrival | actual_arrival |
|-----------|-----------|-------------------|----------------|
| 1         | US-CT     | 2023-09-08        | 2023-11-07     |
| 2         | US-NM     | 2023-09-17        | 2024-01-23     |
| 3         | RU-KR     | 2023-03-18        | 2023-04-07     |
| 4         | US-AZ     | 2023-04-08        | 2023-08-01     |
| 5         | IN-OR     | 2023-09-19        | 2023-12-03     |
| 6         | CA-NL     | 2023-06-04        | 2023-11-17     |
| 7         | BR-PE     | 2023-06-02        | 2023-11-09     |
| 8         | TH-32     | 2023-03-30        | 2023-07-21     |
| 9         | US-MS     | 2024-01-22        | 2024-03-06     |

The status bar at the bottom indicates 'Total rows: 509' and 'Query complete 00:00:00.177'. A green message box at the bottom right states: 'Successfully run. Total query runtime: 177 msec. 509 rows affected.'

7. Find flights that arrived late according to their actual arrival time compared to the scheduled arrival time.

The screenshot shows the pgAdmin 4 interface. The left pane displays the 'airports' database structure, with the 'flights' table selected. The right pane shows a SQL query and its results.

```
select flight_id, flight_no, scheduled_arrival, actual_arrival from flights
where scheduled_arrival < actual_arrival
```

The query results are displayed in a table with 5 columns: flight\_id, flight\_no, scheduled\_arrival, and actual\_arrival. The table shows 20 rows of data, with the first 9 rows visible in the screenshot.

| flight_id | flight_no | scheduled_arrival | actual_arrival |
|-----------|-----------|-------------------|----------------|
| 1         | US-CT     | 2023-09-08        | 2023-11-07     |
| 2         | US-NM     | 2023-09-17        | 2024-01-23     |
| 3         | RU-KR     | 2023-03-18        | 2023-04-07     |
| 4         | US-AZ     | 2023-04-08        | 2023-08-01     |
| 5         | IN-OR     | 2023-09-19        | 2023-12-03     |
| 6         | CAN-L     | 2023-06-04        | 2023-11-17     |
| 7         | BR-PE     | 2023-06-02        | 2023-11-09     |
| 8         | TH-32     | 2023-03-30        | 2023-07-21     |
| 9         | US-MS     | 2024-01-22        | 2024-03-06     |

8. Show airlines from any of: ('France','Portugal','Poland') created between '2023-11-01' and '2024-03-31'.

The screenshot shows the pgAdmin 4 interface. The left pane displays the 'airline' database structure, with the 'airline' table selected. The right pane shows a SQL query and its results.

```
select * from airline
where airline_country in ('France', 'Portugal', 'Poland')
and created_at between '2023-11-01' and '2024-03-31'
```

The query results are displayed in a table with 7 columns: airline\_id, airline\_code, airline\_name, airline\_country, created\_at, and update\_at. The table shows 7 rows of data.

| airline_id | airline_code | airline_name | airline_country | created_at | update_at  |
|------------|--------------|--------------|-----------------|------------|------------|
| 4          | OOKB         | KHS          | France          | 2023-11-18 | 2024-02-18 |
| 13         | OIFE         | IFH          | Portugal        | 2024-01-09 | 2023-11-09 |
| 15         | SYKZ         | KKG          | Poland          | 2023-12-24 | 2023-10-30 |
| 24         | ZSRG         | RUG          | Portugal        | 2024-03-11 | 2023-08-24 |
| 28         | RKPD         | JDG          | Poland          | 2023-11-03 | 2023-12-06 |
| 30         | CYST         | YST          | Poland          | 2023-11-24 | 2023-10-07 |
| 43         | AYKM         | KMA          | France          | 2023-11-21 | 2023-05-30 |

## 9. Find top3 overweighted baggage with more than 25kg.

The screenshot shows the pgAdmin 4 interface. The Object Explorer on the left displays the database structure, with the 'baggage' table selected under the 'airline' schema. The SQL editor on the right contains the following query:

```
1 select * from baggage
2 where weight_in_kg >= 25
3 order by weight_in_kg DESC
4 limit 3
```

The Data Output pane shows the results of the query, displaying 3 rows. The status bar at the bottom indicates 'Total rows: 3' and 'Query complete 00:00:00.076'.

| baggage_id [PK] integer | weight_in_kg numeric (4,2) | created_date date | update_date date | booking_id integer |
|-------------------------|----------------------------|-------------------|------------------|--------------------|
| 321                     | 49.80                      | 2023-07-07        | 2023-11-10       | 349                |
| 61                      | 49.57                      | 2023-04-21        | 2023-07-31       | 464                |
| 102                     | 49.44                      | 2024-01-24        | 2023-11-01       | 53                 |

## 10. Find the youngest passengers' full name.

The screenshot shows the pgAdmin 4 interface. The Object Explorer on the left displays the database structure, with the 'passengers' table selected under the 'airline' schema. The SQL editor on the right contains the following query:

```
1 select first_name, last_name from passengers
2 order by date_of_birth desc
3 limit 1
```

The Data Output pane shows the results of the query, displaying 1 row. The status bar at the bottom indicates 'Total rows: 1' and 'Query complete 00:00:00.079'.

| first_name character varying (50) | last_name character varying (50) |
|-----------------------------------|----------------------------------|
| Bernie                            | Michal                           |

11. Find the cheapest booking price on each booking platform.

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including the 'booking' table. The central pane contains the following SQL query:

```
1 select booking_platform, MIN(price) as cheapest_price from booking
2 group by booking_platform
3
```

The bottom pane shows the query results in a table with two columns: 'booking\_platform' and 'cheapest\_price'. The results are as follows:

| booking_platform               | cheapest_price |
|--------------------------------|----------------|
| Schmitt-Quigley                | 4966.58        |
| Rohan, Hamill and Bosco        | 7118.84        |
| Kutch-Russel                   | 2113.46        |
| Veum-Corwin                    | 2183.14        |
| Wolff, Wuckert and Satterfield | 4911.89        |
| Reichel, Deckow and Quitzon    | 8389.34        |
| Murazik-Gulgowski              | 950.80         |
| Cole, Corwin and Leannon       | 8735.70        |
| Franecki, Dickens and Walsh    | 283.78         |

A status message at the bottom indicates: 'Successfully run. Total query runtime: 77 msec. 494 rows affected.'

12. Return airlines whose airline\_code contains a digit.

The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including the 'airline' table. The central pane contains the following SQL query:

```
1 select * from airline
2 where airline_code ~ '[0-9]'
```

The bottom pane shows the query results in a table with columns: 'airline\_id', 'airline\_code', 'airline\_name', 'airline\_country', 'created\_at', and 'update\_at'. The results are as follows:

| airline_id | airline_code | airline_name | airline_country | created_at | update_at  |
|------------|--------------|--------------|-----------------|------------|------------|
| 22         | 2AK6         | HGZ          | Philippines     | 2024-03-13 | 2023-10-20 |

A status message at the bottom indicates: 'Query complete 00:00:00.083'.

13. List the top5 most recently created airlines.

The screenshot shows the pgAdmin 4 interface. The Object Explorer on the left displays the database structure, including the 'airline' table. The SQL query editor in the center contains the following query:

```
1 select * from airline
2 order by created_at DESC
3 limit 5
```

The Data Output pane at the bottom shows the results of the query, displaying 5 rows of airline data:

| airline_id | airline_code | airline_name | airline_country | created_at | update_at  |
|------------|--------------|--------------|-----------------|------------|------------|
| 9          | KNQX         | NQX          | Venezuela       | 2024-03-16 | 2023-09-16 |
| 40         | LTBD         | Cit          | Argentina       | 2024-03-15 | 2024-01-29 |
| 22         | 2AKG         | HGZ          | Philippines     | 2024-03-13 | 2023-10-20 |
| 24         | ZSRG         | RUG          | Portugal        | 2024-03-11 | 2023-08-24 |
| 45         | HCMV         | BUO          | Sweden          | 2024-02-18 | 2023-12-10 |

A status message at the bottom indicates: "Successfully run. Total query runtime: 80 msec. 5 rows affected."

14. Return all rows where booking\_id is between 200 and 300 inclusive and check\_result <> 'Checked'.

The screenshot shows the pgAdmin 4 interface. The Object Explorer on the left displays the database structure, including the 'booking' and 'baggage\_check' tables. The SQL query editor in the center contains the following query:

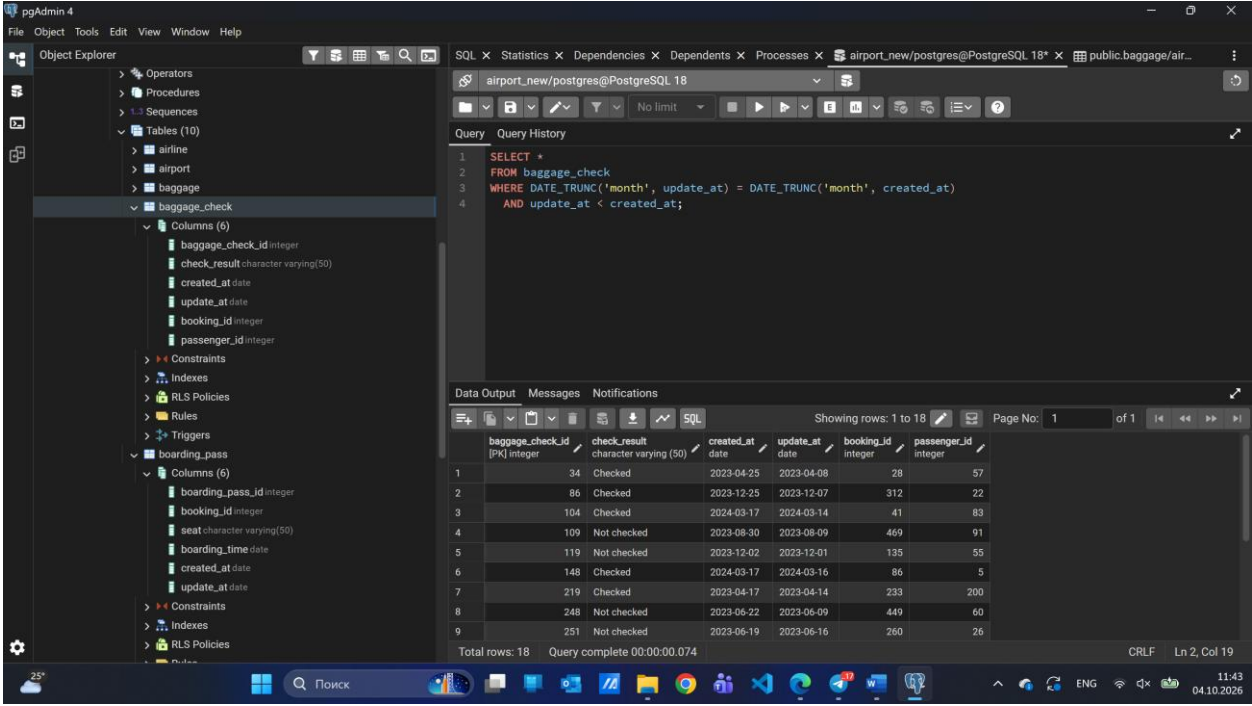
```
1 select * from booking
2 where booking_id IN (
3   select booking_id from booking
4   where booking_id BETWEEN 200 and 300
5   INTERSECT
6   select booking_id from baggage_check
7   where check_result <> 'Checked'
8 )
```

The Data Output pane at the bottom shows the results of the query, displaying 41 rows of booking data:

| booking_id | passenger_id | booking_platform                  | created_at | update_at  | status | price   |
|------------|--------------|-----------------------------------|------------|------------|--------|---------|
| 204        | 2            | Weber, Cruickshank and Rosenba... | 2024-02-12 | 2024-02-16 | Male   | 3087.02 |
| 210        | 60           | Stroman-O'Reilly                  | 2024-02-22 | 2023-04-29 | Female | 9162.75 |
| 218        | 28           | Conroy, Wilderman and Bradtke     | 2023-05-25 | 2023-06-19 | Female | 1703.15 |
| 225        | 68           | Beer, Emmerich and Wehner         | 2023-10-28 | 2024-01-08 | Male   | 6960.34 |
| 226        | 154          | Johnston-Osinski                  | 2023-07-18 | 2023-07-21 | Male   | 2407.54 |
| 231        | 62           | Harber, Emmerich and Connelly     | 2023-04-29 | 2024-01-17 | Female | 9395.39 |
| 232        | 23           | Roob-Yundt                        | 2023-04-15 | 2023-08-24 | Female | 4316.52 |
| 234        | 81           | Kuhic-Walker                      | 2023-05-07 | 2024-01-31 | Female | 6229.89 |
| 237        | 35           | Fahey-Becker                      | 2023-12-31 |            |        |         |

A status message at the bottom indicates: "Successfully run. Total query runtime: 73 msec. 41 rows affected."

15. Baggage checks where update\_at is in the same month as created\_at but occurs earlier than created\_at.



The screenshot shows the pgAdmin 4 interface. The left pane displays the database structure, including the 'baggage\_check' table. The right pane shows a SQL query and its results.

**SQL Query:**

```
1 SELECT *
2 FROM baggage_check
3 WHERE DATE_TRUNC('month', update_at) = DATE_TRUNC('month', created_at)
4 AND update_at < created_at;
```

**Data Output:**

| baggage_check_id [PK] integer | check_result character varying (50) | created_at date | update_at date | booking_id integer | passenger_id integer |
|-------------------------------|-------------------------------------|-----------------|----------------|--------------------|----------------------|
| 34                            | Checked                             | 2023-04-25      | 2023-04-08     | 28                 | 57                   |
| 86                            | Checked                             | 2023-12-25      | 2023-12-07     | 312                | 22                   |
| 104                           | Checked                             | 2024-03-17      | 2024-03-14     | 41                 | 83                   |
| 109                           | Not checked                         | 2023-08-30      | 2023-08-09     | 469                | 91                   |
| 119                           | Not checked                         | 2023-12-02      | 2023-12-01     | 135                | 55                   |
| 148                           | Checked                             | 2024-03-17      | 2024-03-16     | 86                 | 5                    |
| 219                           | Checked                             | 2023-04-17      | 2023-04-14     | 233                | 200                  |
| 248                           | Not checked                         | 2023-06-22      | 2023-06-09     | 449                | 60                   |
| 251                           | Not checked                         | 2023-06-19      | 2023-06-16     | 260                | 26                   |

Total rows: 18 Query complete 00:00:00.074 CRLF Ln 2, Col 19